

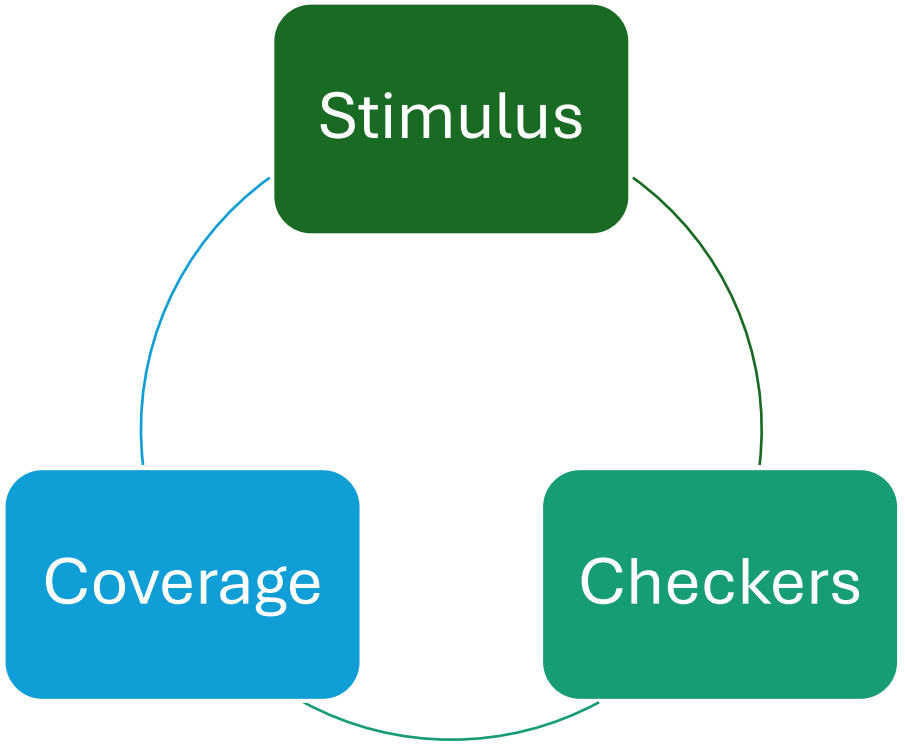
Verification Concepts

Aman Arora

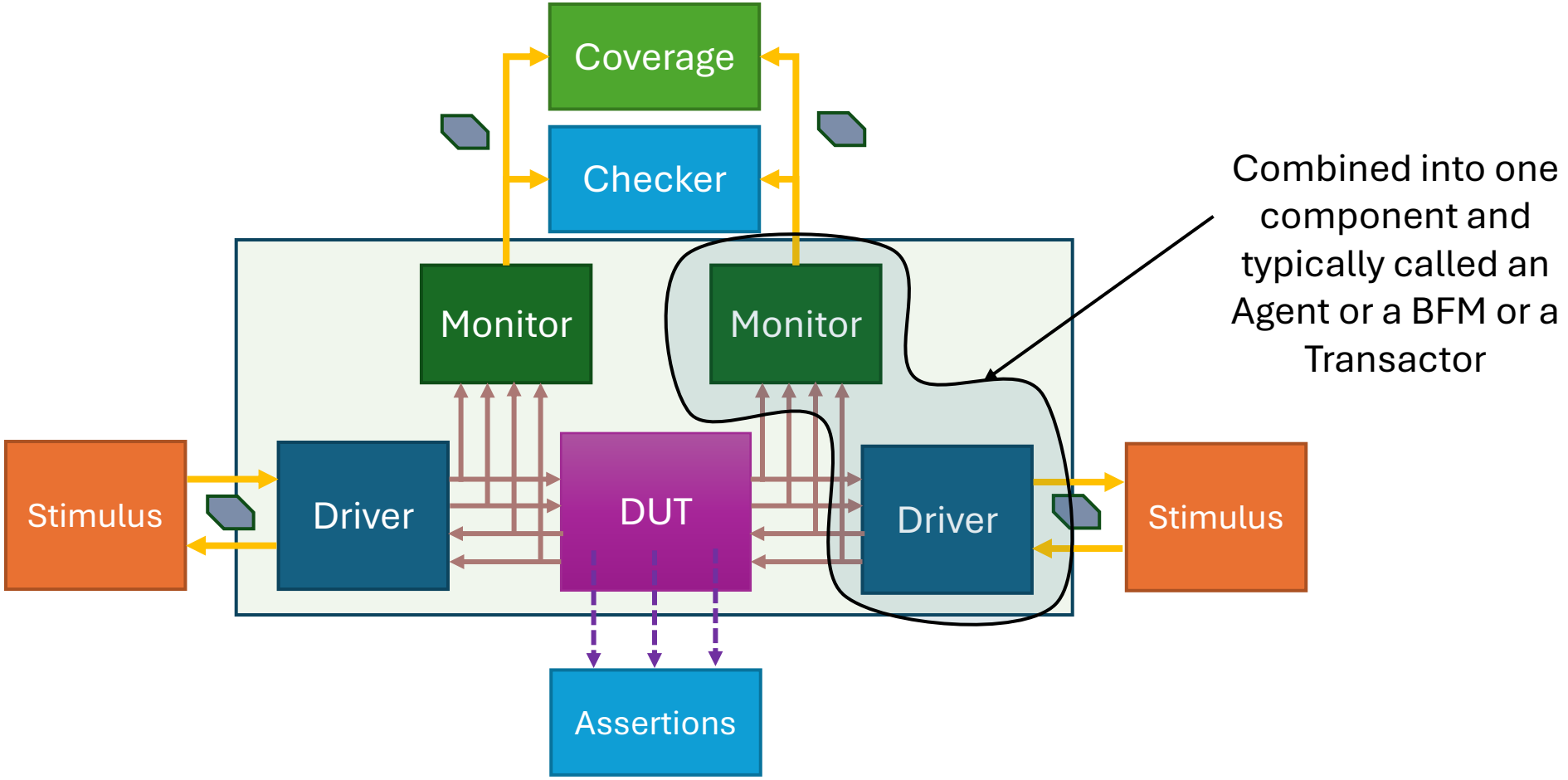
Verification Plan

- A document that describes how will you verify a DUT
- Strategy
  - Methodology
  - Simulation, Formal
  - Testbench components
  - Sign-off metrics
- Features
  - For every feature:
    - Stimulus (“Apply inputs A➔B➔C, keeping D=0”)
    - Checking (“Check that E becomes 1 every time we enter state X”)
    - Coverage (“State X was reached”)
- Multiple formats – Word, Excel, Synopsys HVP, Cadence VPlan

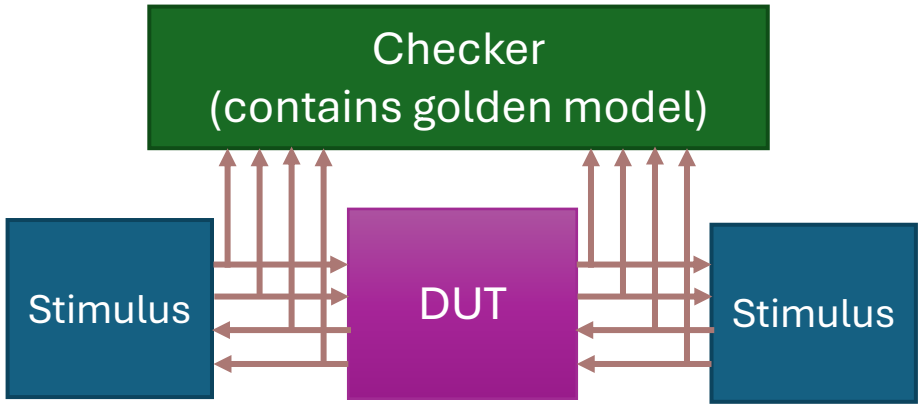
The trifecta of verification



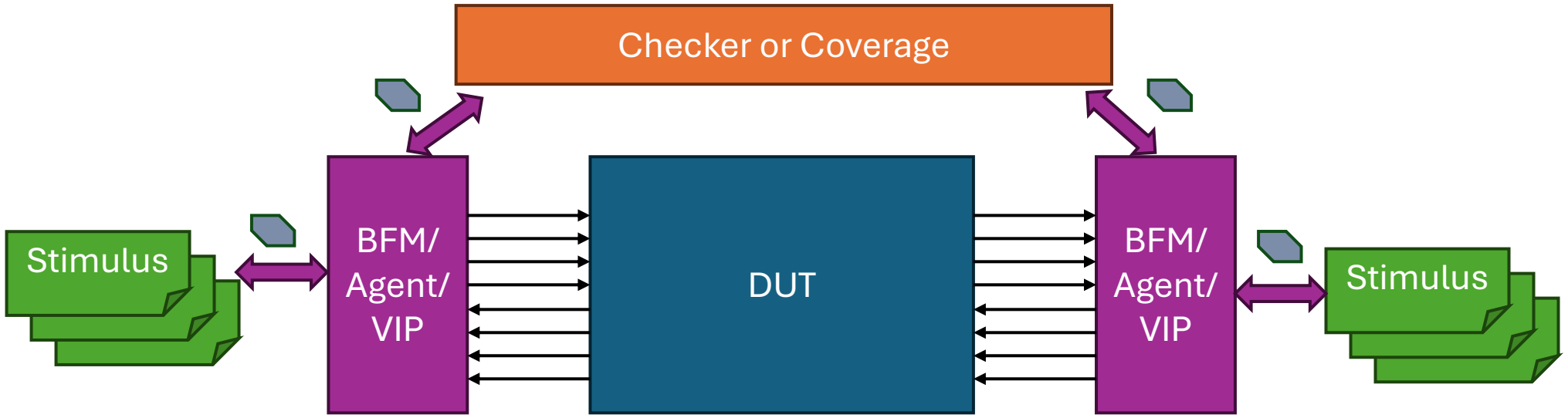
Testbench Architecture



Testbench Architecture



Testbench Architecture



Putting together software (“classes”) and hardware (“DUT”)

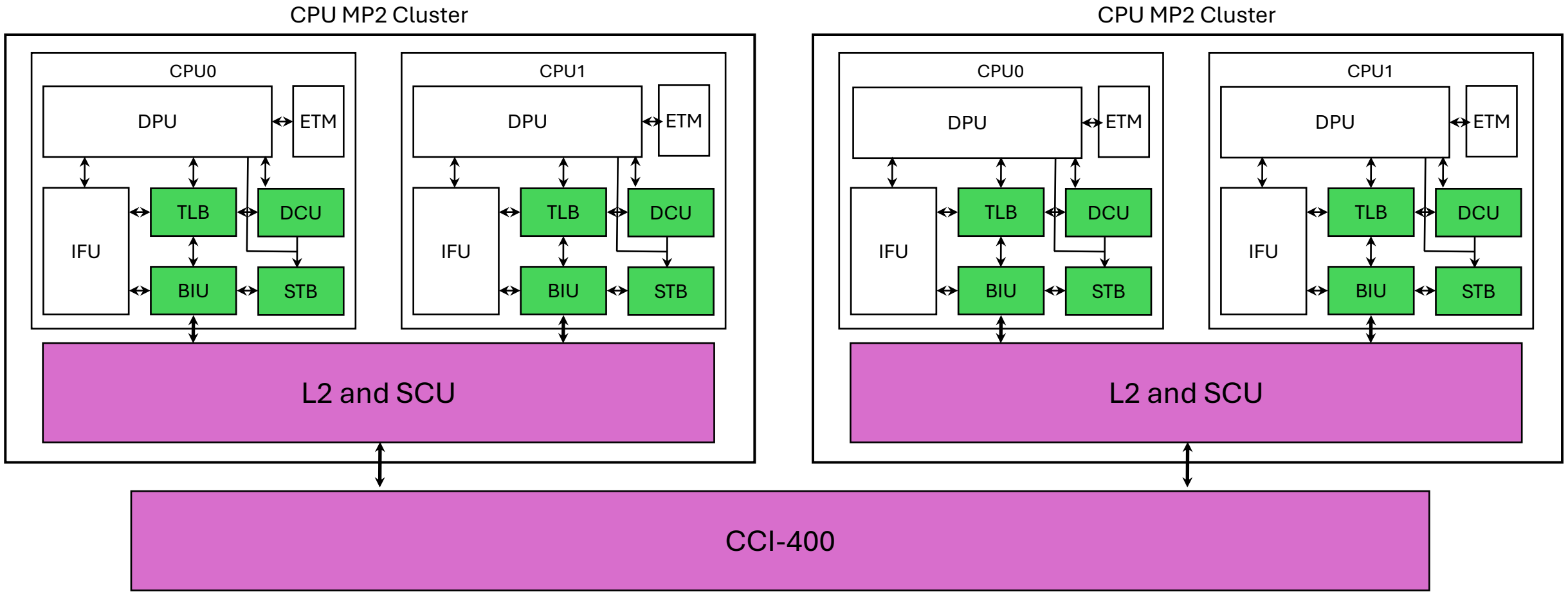
- Start with simple SystemVerilog testbench (interface, class, dut)
- Add package
- Add transaction
- Add randomization
- Add transactors/BFM
- Add monitor
- Add covergroup and sampling
- Add assertion
- Add fork/join

www.edaplayground.com

Hierarchical Verification

- Unit-level
- Sub-system level
- Top-level

- Reuse
- Emulate another block that would have been on that interface.
  - Could have used the RTL or a simpler version of the unit. But that may not be ready or may be buggy.
  - You only need the interface behavior anyway. Using an agent provides more controllability and observability.


CPU MP2 Cluster
CPU0
DPU
ETM
IFU
TLB
DCU
BIU
STB
CPU1
DPU
ETM
IFU
TLB
DCU
BIU
STB
L2 and SCU
CPU MP2 Cluster
CPU0
DPU
ETM
IFU
TLB
DCU
BIU
STB
CPU1
DPU
ETM
IFU
TLB
DCU
BIU
STB
L2 and SCU
CCI-400

Synopsys Training

- Please complete this training: SystemVerilog for Verification

Stimulus

Randomization

- Directed tests
- Randomized tests
- Constrained random tests

- Corner cases
  - Hard-to-hit
  - Still use directed tests for these

Inline vs. explicit constraints

- assert(t.randomize() with {addr >= 50; addr <= 1500; data < 10;});

- constraint c_short {length inside {[1:32]}; }

Rand mode and Constraint mode

- Generating illegal/error stimulus

Active and passive/reactive stimulus

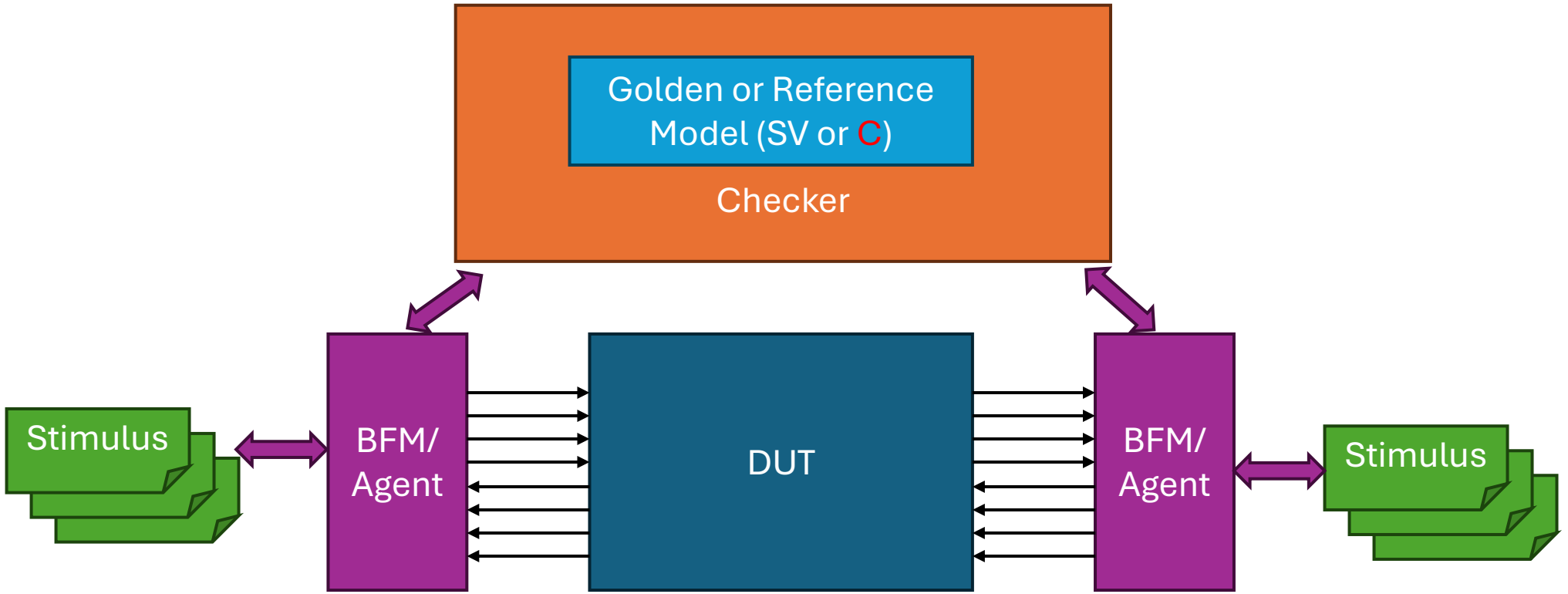
Irritators

Checking

Types

- Self-Checking Stimulus
- Assertions
- Checkers
  - Scoreboards

Golden or Reference Models

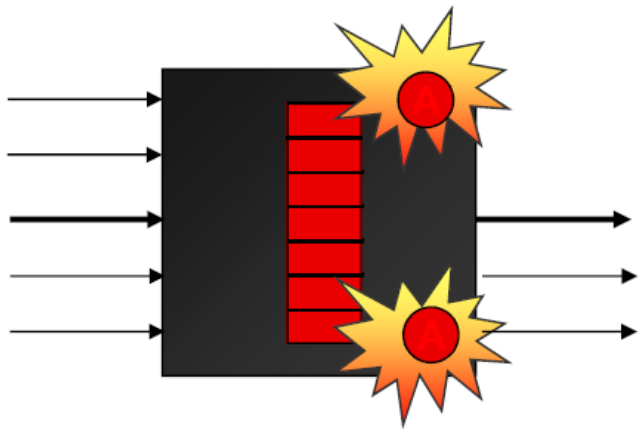


Model accuracy and granularity

- Cycle-accurate
- Boundary-accurate

Assertions

- An assertion is a directive that a particular property is required to be true
- A property is a statement specifies behavior or intent



// Assert that the FIFO controller
// cannot overflow nor underflow

Types of Assertions - 1

- **Combinatorial** (i.e. “zero-time”) **conditions**
  - ensure functional correctness
  - must be valid at all times
    - “The buffer never overflows.”
    - “The register always holds a single-digit value.”
    - “The state machine encoding is *one hot*.”
- **Temporal conditions**
  - to verify sequential functional behaviour over a period of time
    - “The grant signal must be asserted for a single clock cycle.”
    - “A request must always be followed by a grant or an abort within 5 clock cycles.”

Types of Assertions - 2

- Safety: Something bad does not happen
  - The FIFO does not overflow.
  - The system does not allow more than one process at a time to modify the shared memory.
  - Requests are answered within 5 clock cycles.

  -- Safety properties can be falsified by a finite simulation run

- Liveness: Something good eventually happens
  - The decoding algorithm eventually terminates.
  - Every request is eventually acknowledged.

  -- Ideally, liveness properties can only be falsified by an infinite simulation run. Practically, we often assume that the “end-of-sim” represents infinite time

Types of Assertions - 3

- High-level
  - Architectural, micro-architectural
  - Typically written by architects in golden models or verification engineers in the testench
- Low-level
  - Implementation level
  - Typically written by designers inside RTL

Arbiter Example

- Consider the requirement for the following arbiter
  - **A requesting client will eventually be served**
    - **We want to ensure that to grants are mutually exclusive**



Arbiter Example

```
a_mutex:
    assert  property (@(posedge clk) disable iff (reset_n)
                                        !(gnt[0] & gnt[1]) );
```



Arbiter Example

- Consider the requirement for the following arbiter
  - **A requesting client will eventually be served**
    - **We want to ensure that a client cannot be starved from access to a shared resource**



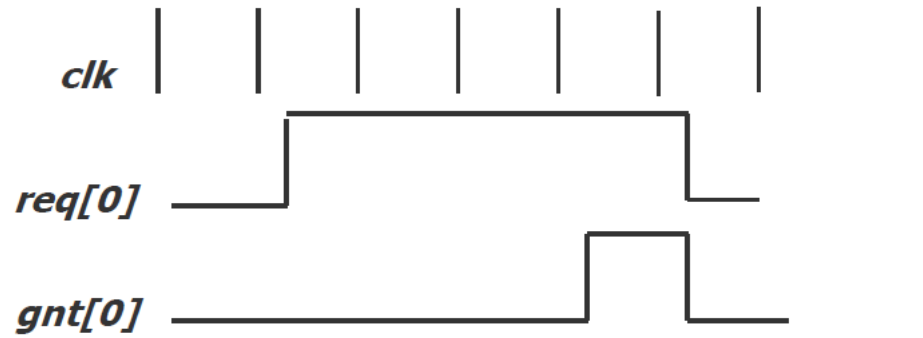
Arbiter Example

```
a_0_not_starved:
    assert  property (@(posedge clk) disable iff (reset_n)
                                    $rose(req[0]) |-> ##[0:$] gnt[0]);
```



Arbiter Example

```
a_0_not_starved:
    assert  property (@(posedge clk) disable iff (reset_n)
                                    $rose(req[0]) |-> ##[0:$] gnt[0]);
```



Trace on which Assertion *a_0_not_starve* holds

Arbiter Example

- We now refine our arbiter assertion to make it more precise
  - **To ensure that an arbiter servicing multiple clients is fair**
- To be fair, a pending request for a particular client should never have to wait more than two arbitration cycles
  - **Otherwise, the alternate was unfairly issued multiple times**



All my assertions pass – now what?

- **An assertion has never "fired".**
  - What does this mean?
  - Does not necessarily mean that it can't be violated!
    - **Unless simulation is exhaustive...,**
      which in practice it never will be.
  - It might not have fired **because it was never active.**

Assertions can either fire & pass, fire & fail, or never fire (vacuous pass).

Bring in assertion coverage!

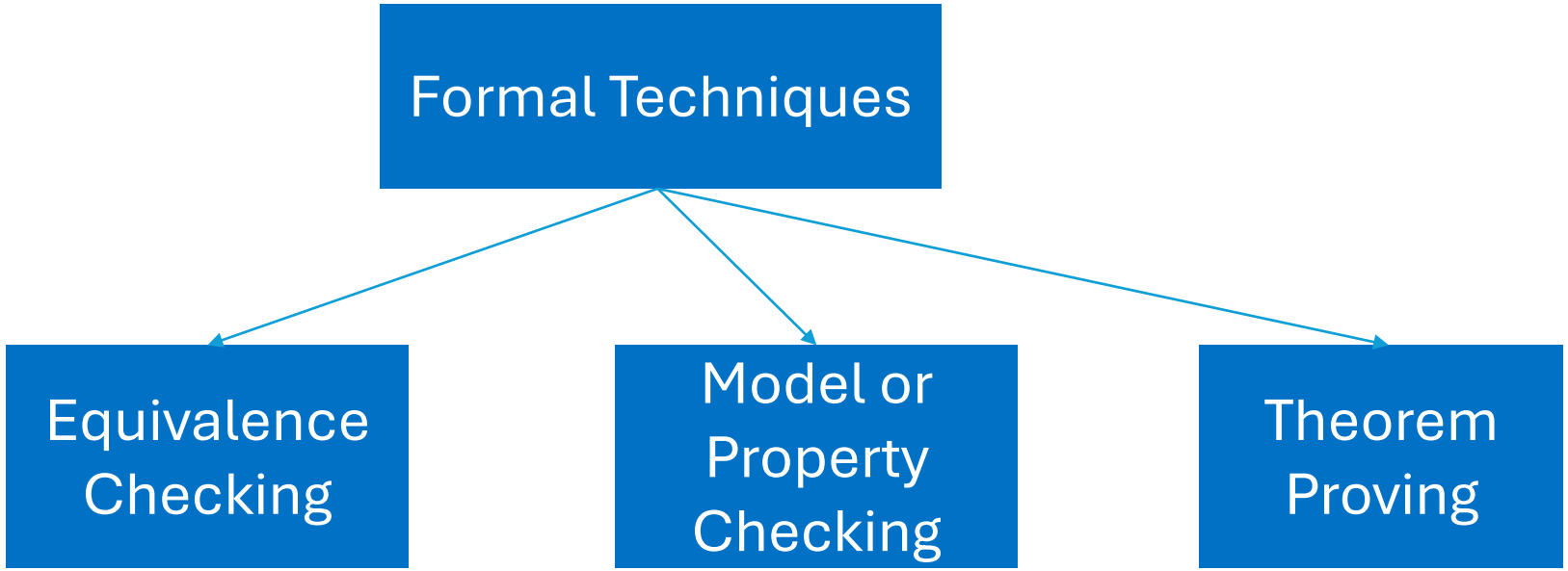
```
assert property ( (sel1 || sel2) |=> ack );
```

```
cover property  ( sel1 || sel2 );
```

System Verilog Assertions (SVA)

Please complete:
https://training.synopsys.com/learn/courses/81/language-systemverilog-assertions

Recall: Formal Verification (FV)



FV mathematically analyze the space of possible behaviors, rather than computing results for specific values. FV covers the whole space, as opposed to simulation which only targets specific values/scenarios.

Assertions/Properties: Simulation vs. Formal

Statement Type	Simulation	FPV
Assertion	Report if any test vectors used in current tests have violated the assertion	Report if it is possible for any set of test vectors to violate the assertion
Assumption	Report if any test vectors used in current tests have violated the assumption	Use the assumptions to limit the space of possible test vectors
Cover	Report how many test vectors used in current tests have exercised the cover property	Report if it is possible for any set of test vectors to exercise the cover property

```
property valid_input_range;
  @(posedge clk) (input_signal >= 0 && input_signal <= 255);
endproperty

// Assume the property holds
assume property (valid_input_range);
```

Assertions/Properties: Simulation vs. Formal

Area of difference	Simulation	FPV
Type and Size of model	Up to full chip, both synthesizable and behavioral	Unit or cluster, or full chip with lot of removed logic, & synthesizable code only
How to reach targeted behaviors	Describe the journey: generate i/p values	Describe the destination: specify target property or state, tool finds if it's reachable
What values are checked	Specific values or set of constrained values	All possible: legal under defined constraints
How do we constrain the model	Active constraints: restrict the stimulus	Passive constraints: Use assumptions to rule out illegal
What do we use for debug	Single Trace	Example trace
How long are the traces	Thousands of cycles – includes start up & do random things to hit complex bug	Tens of cycles: FPV generates minimal path to target

Coverage

Code Coverage

- Statement/line coverage
  - Did this line execute?
- Branch coverage
  - Did both sides of the if execute?
- Toggle coverage
  - Did bits/signals toggle?
- FSM coverage
  - Did we visit states and exercise transitions?

Code Coverage Limitations

Functional Coverage

- Especially important with randomized stimulus
- If we were only using directed stimulus, then coverage isn’t necessary

Crosses

- ignore_bins
- illegal_bins

Sampling Covergroups

- Where to sample is important

Coverage Merge

- .. And do not count coverage for failing tests

Exclusion Lists